

The Query Language, Explained by Example

What the old languages do, where they break, and what we do instead
(language name TBD; syntax shown is the current working sketch)

LossMetricKG project

August 24, 2026

Abstract

This document teaches the proposed agent graph language the way the evidence says languages should be taught: through worked examples. Part I sets up one running example graph and asks the same questions of the five languages that matter today (SQL, Cypher, SPARQL, Gremlin, and the new ISO standard GQL), showing what each does well and exactly where language models break when writing it. Part II rebuilds the same questions in the new language, one feature at a time, so every design choice is visible as a diff against something familiar. Part III covers what no existing query language has at all: structural writes with receipts, deduplication surfacing, granularity refinement, health queries, and token budgets. A final session transcript shows an agent doing real construction work end to end.

Contents

1	The running example	3
2	Part I: the old languages	3
2.1	SQL: the relational baseline	3
2.2	Cypher: the incumbent graph language	4
2.3	SPARQL: the semantic-web veteran	4
2.4	Gremlin: the traversal machine	5
2.5	GQL and SQL/PGQ: the new ISO standards	5
2.6	Scorecard	6
3	Part II: the same questions, rebuilt	6
3.1	One line, one step, one name	7
3.2	Role-named edges: direction cannot be written	7
3.3	Schema-closed vocabulary	8
3.4	Staged, explicit aggregation	8
3.5	Canonical forms	8
3.6	Verify before execute	9
3.7	Token budgets and continuation	9
3.8	The result format is part of the language	9

4	Part III: what no old language has at all	10
4.1	Structural writes with receipts	10
4.2	Deduplication: system detects, agent resolves	11
4.3	Granularity: blob, refine, compact	11
4.4	Health and failure nodes	12
4.5	Schema evolution: derive class	12
5	A construction session, end to end	12
6	Feature map: old world to new	14

1 The running example

One graph, used for every example in this document. A small supply-chain world:

- **Node classes:** `product` (name, launch_year), `part` (name, unit_cost), `supplier` (name, country).
- **Edge types with named roles:** `uses` connects a `product` (role: *whole*) to a `part` (role: *component*); `supplied_by` connects a `part` (role: *item*) to a `supplier` (role: *source*).

And two questions, one easy and one hard, asked of every language:

Q1 (one hop): Which suppliers supply the part called “lithium cell”?

Q2 (multi-hop + aggregation): For products launched after 2024, which suppliers provide their parts, and how many distinct parts does each supplier provide?

Q1 is the kind of question where familiar languages are fine and a new language cannot win much; Q2 is the compositional kind where roughly 40% of frontier-model queries fail today and where the Anka result says a designed language wins big. Keep that asymmetry in mind throughout: the new language is built for Q2-shaped work.

2 Part I: the old languages

2.1 SQL: the relational baseline

The graph lives in three entity tables plus two join tables (`uses`, `supplied_by`) holding foreign keys. Q2:

```
SELECT s.name, COUNT(DISTINCT pt.id) AS n_parts
FROM product p
JOIN uses u      ON u.product_id = p.id
JOIN part pt     ON pt.id        = u.part_id
JOIN supplied_by sb ON sb.part_id = pt.id
JOIN supplier s  ON s.id         = sb.supplier_id
WHERE p.launch_year > 2024
GROUP BY s.id, s.name;
```

Features worth stealing. Explicit `GROUP BY` (you can see the grouping key), a huge training corpus (673k Stack Overflow posts, which is why LLMs write SQL better than any graph language), and declarative optimizer-friendly semantics. Note the `GROUP BY s.id, s.name`: grouping by `s.name` alone would silently merge two suppliers that share a name. Identity-versus-value grouping is a real trap in every language in this part, and one models fall into constantly.

Features that hurt. Every hop is a join the model must assemble by hand from foreign-key knowledge; a four-join query has four chances to pair the wrong columns. Multi-hop paths of unknown length (“parts connected through any chain of subassemblies”) are painful recursive CTEs. The graph structure is implicit in the schema, invisible in the query.

Where models break here

Join-condition errors dominate: wrong column pairings that still typecheck and run, returning plausible garbage. This is the relational cousin of the reversed-edge failure. Silent, hard to catch without checking answers.

2.2 Cypher: the incumbent graph language

Neo4j’s language, the one all the agent benchmarks target. Patterns are drawn as ASCII art: nodes in parentheses, edges in brackets, direction as an arrow.

```
MATCH (p:Product)-[:USES]->(pt:Part)-[:SUPPLIED_BY]->(s:Supplier)
WHERE p.launch_year > 2024
WITH s, count(DISTINCT pt) AS n_parts
RETURN s.name, n_parts
```

Features worth stealing. The pattern reads like a picture of the path, one line expresses what took SQL five clauses, and variable-length paths are trivial (`-[:USES*1..3]->`). Largest corpus of any graph language.

Features that hurt. Three of the five documented LLM failure modes live in this one query. The arrows: is it `(pt)-[:SUPPLIED_BY]->(s)` or `(s)-[:SUPPLIED_BY]->(pt)`? The model must recall the schema’s direction convention, and reversed direction is the single most-cited text-to-Cypher failure; entire repair systems exist just to flip arrows. The aggregation: the natural one-liner `RETURN s.name, count(DISTINCT pt)` groups *implicitly* by every non-aggregated return column, here by supplier *name*, silently merging same-named suppliers; the correct query needs the `WITH s` step above to group by node identity, and changing a return list silently changes the grouping. Cypher 25 added an optional explicit `GROUP BY`, but the implicit form remains legal, so the trap persists and there are now two spellings of the same thing, the opposite of a canonical form. And label hallucination: nothing stops the model emitting `(:Vendor)` when the schema says `Supplier`; the query fails, or worse, matches nothing and returns an empty answer that looks like a fact.

Where models break here

CypherBench error taxonomy on exactly this query shape: reversed `SUPPLIED_BY` arrow (most common), hallucinated `Vendor/Component` labels, and implicit-grouping mistakes when a second property is added to `RETURN`. Best frontier models: about 60% execution accuracy overall, worse on multi-hop slices [arXiv 2412.18702].

2.3 SPARQL: the semantic-web veteran

The W3C standard for RDF data, where everything, including the schema, is triples of **subject predicate object**.

```
PREFIX : <http://example.org/supply#>
SELECT (SAMPLE(?sname) AS ?name) (COUNT(DISTINCT ?pt) AS ?nparts)
WHERE {
  ?p a :Product ;   :launchYear ?y ;   :uses ?pt .
  ?pt :suppliedBy ?s .
  ?s  :name ?sname .
  FILTER(?y > 2024)
}
GROUP BY ?s
```

Features worth stealing. Triple patterns are wonderfully uniform (there is only one syntactic shape in the whole language), federation across datasets is built in, and the explicit `GROUP BY` returns.

Features that hurt. URI ceremony everywhere; the `PREFIX` machinery and `?variable` sigils burn tokens and create failure surface. The corpus is tiny (6k Stack Overflow posts), and it

shows: most models score under 4% zero-shot on SPARQL generation [arXiv 2411.05521]. RDF serialization is also the most token-expensive way known to show a graph to a model, 3 to 5 times the cost of an edge list for zero accuracy gain [arXiv 2504.07087].

Where models break here

Predicate direction again (`?pt :suppliedBy ?s` vs `?s :supplies ?pt`, both plausible), invented predicates, and punctuation errors: the semicolon/period/comma continuation rules inside `WHERE` are exactly the kind of long-range, low-redundancy syntax models fumble.

2.4 Gremlin: the traversal machine

Apache TinkerPop’s language. Not declarative: a Gremlin query is an imperative pipeline of traversal steps, executed left to right.

```
g.V().hasLabel('product').has('launchYear', gt(2024)).
  out('uses').dedup().
  out('suppliedBy').
  groupCount() // keys are supplier vertices, not names
```

Features worth stealing. Composability: each step is small and typed, and the pipeline shape means you can reason about one hop at a time. This is the closest ancestor of our line-oriented design.

Features that hurt. Everything is method chaining on one expression: no named intermediates, so the model must track the implicit “current position” of the traverser through a dozen chained calls. The Anka study measured exactly this pattern (chaining plus shadowing) as the cause of 69% of failures in Python pipelines. `out()` vs `in()` is the direction failure again, now spelled as method names. The placement of `dedup()` changes the count semantics silently. And the tempting `.by('name')` modulator on `groupCount` groups by name string, merging same-named suppliers; grouping by vertex is correct but then extracting names needs a further `unfold().project(...)` chain.

Where models break here

Wrong `out/in` choice, lost track of traverser position mid-chain, `dedup()` in the wrong spot producing subtly wrong counts. Chaining is the named-intermediates failure mode in its purest form.

2.5 GQL and SQL/PGQ: the new ISO standards

GQL (ISO/IEC 39075:2024) standardises the Cypher pattern style; SQL/PGQ embeds the same patterns inside SQL via `GRAPH_TABLE`. Q2 in SQL/PGQ:

```
SELECT sname, COUNT(DISTINCT ptid) AS n_parts
FROM GRAPH_TABLE (supply_graph,
  MATCH (p IS product)-[IS uses]->(pt IS part)
        -[IS supplied_by]->(s IS supplier)
  WHERE p.launch_year > 2024
  COLUMNS (s.id AS sid, s.name AS sname, pt.id AS ptid))
GROUP BY sid, sname;
```

Features worth stealing. Explicit per-query path semantics (`WALK/TRAIL/ACYCLIC` keywords,

a genuinely good idea we simplify rather than copy), and standardisation gravity: this will be in Postgres, Oracle, DuckDB, everywhere.

Features that hurt. It inherits Cypher’s arrows and implicit grouping wholesale, adds SQL’s ceremony around them, and, being new, has almost no corpus: GQL currently benchmarks *worse* than Cypher for LLM generation purely from unfamiliarity [arXiv 2602.11745]. Standard-language status does not buy generation accuracy.

Where models break here

All Cypher failure modes, plus new-syntax unfamiliarity. The instructive part: GQL proves that corpus familiarity, not language quality, sets today’s accuracy, which is exactly the variable a designed language plus in-context teaching attacks.

2.6 Scorecard

	SQL	Cypher	SPARQL	Gremlin	GQL/PGQ
Multi-hop ergonomics	poor	good	fair	good	good
Direction mistake possible	(joins)	yes	yes	yes	yes
Grouping explicit	yes	optional	yes	no	optional
Named intermediates	CTEs only	WITH	vars	no	LET
Schema closed (can’t hallucinate)	no	no	no	no	no
Line-local syntax	fair	no	no	no	no
Token cost of results	n/a	mixed	worst	mixed	mixed
LLM corpus	huge	mid	tiny	small	none
Agent write surface (receipts, dedup)	no	no	no	no	no

Two rows need honesty notes. “Optional” grouping: Cypher 25 and GQL both added explicit `GROUP BY`, but the implicit form stays legal in each, so the trap survives and the language gains a second spelling. And every language here *has* write operations (`INSERT`, `CREATE`, `SPARQL Update`, `addV`); what none has is a write surface built for agents: provenance-carrying verbs, receipts, dedup candidates, lineage. That distinction is what the last row measures. No column has the profile we need; the bottom rows are the design brief for Part II.

Plain words

Declarative: you state what you want, the engine picks how (SQL, Cypher). **Imperative/traversal:** you state the steps (Gremlin). **RDF/triples:** data model where every fact is subject-predicate-object. **CTE:** SQL’s `WITH` clause for naming intermediate results. **Corpus:** the amount of code in a language the model saw during training; the dominant predictor of generation accuracy today.

3 Part II: the same questions, rebuilt

Now the new language, feature by feature. Each feature is introduced as a diff against a failure you just saw. First the full answers, then the anatomy.

Q1, one hop:

```
find part where name = "lithium cell" as cell
follow cell supplied_by source as sups
return sups.name
```

Q2, multi-hop with aggregation:

```
find product where launch_year > 2024 as recent
follow recent uses component as parts
follow parts supplied_by source as sups
group by sups as g
count distinct g.parts as n_parts
return sups.name, n_parts order by n_parts budget 2000 tokens
```

How to read this: a query builds one **binding table**, in the style of SPARQL solution mappings. Each name (**recent**, **parts**, **sups**) is a column; each **follow** extends the rows, keeping provenance, so a row remembers *which* product led to *which* part led to *which* supplier. **group by sups** groups rows by the *node identity* bound in that column, never by a display property; two suppliers who share a name stay separate. Grouping by a value is a visibly different spelling (**group by sups.country**), so the identity-versus-value choice that silently corrupts the Part I examples is explicit here. **count distinct g.parts** counts distinct part bindings within each group. There is no **limit**: the query answers the whole question, and the **budget** clause handles size with explicit truncation and continuation (Section 3.7). Now the anatomy.

3.1 One line, one step, one name

New feature: line-oriented named steps

Every line performs exactly one operation and binds its result to exactly one name. No chaining, no nesting, no brackets to balance across distance. The Gremlin pipeline above becomes five lines, each inspectable.

Compare the Gremlin chain, where the traverser position is implicit, with the rewrite: **recent**, **parts**, **sups**, **g**, **n_parts**. If the model mis-thinks step three, the error is localised to one line mentioning named inputs, and the verifier can report “**sups** is a set of suppliers; you grouped by **part**” instead of “query returned wrong answer”. This is the direct application of the Anka finding: named intermediates eliminated the failure class that caused 69% of Python pipeline errors. Locality also kills the bracket-matching failure mode: there are no delimiters that open on one line and close on another.

3.2 Role-named edges: direction cannot be written

New feature: role traversal

Edges are traversed by naming the *role* you want to arrive at, never by drawing a direction. **follow parts supplied_by source** reads: from **parts**, across **supplied_by** edges, to the node playing the *source* role. The reverse trip is **follow sups supplied_by item**, a different word, not a flipped arrow.

Recall the Cypher failure: (pt)-[:SUPPLIED_BY]->(s) vs the reversed arrow, both syntactically valid, one silently wrong. Here the schema declares **supplied_by(item: part, source: supplier)**, and the query mentions the destination role by name. Honest accounting of what this buys. When the two endpoints have different classes, as here, naming the wrong role is a *type error*: **follow parts supplied_by item** asks to arrive at the role **parts** already occupies, and the verifier rejects the plan before execution with a corrective message. When both endpoints share a class (**manages(manager: person, report: person)**), the role choice is genuinely semantic and no grammar can decide it for the model; what the design changes is that

the choice is carried by a meaningful word (**manager** vs **report**) rather than a direction glyph, and words are where the language prior actually helps. So the most-cited text-to-Cypher failure mode is machine-checked in the heterogeneous case and made legible in the homogeneous one, not eliminated outright.

3.3 Schema-closed vocabulary

New feature: schema-closed grammar

The set of legal class names, edge types, role names, and properties in any query is generated from the live schema. With constrained decoding, the model *cannot emit* **vendor** if the schema says **supplier**: the token is not in the grammar. Without constrained decoding, the same closure runs as parse-time validation with corrective errors.

The Cypher hallucination failure (**:Vendor**, **:Component**) dies at decode time. When constrained decoding is unavailable (closed APIs), the verifier catches it before execution and answers in natural language built for self-repair:

```
error: unknown class "vendor" in line 1
  did you mean: supplier (class), or distributor (class)?
  schema excerpt: part --supplied_by--> supplier {name, country}
nothing was executed.
```

The error message is a language feature, not an afterthought: execution-error feedback is the only form of self-correction that measurably works, so errors name the line, offer schema-grounded alternatives, and confirm nothing ran.

3.4 Staged, explicit aggregation

New feature: group then count, as separate named steps

Aggregation is two lines: **group by** <column> **as** <name> declares the grouping key explicitly (a bound name groups by node identity; <column>.<property> groups by value, a visibly different spelling), then **count/sum/...** consumes the groups. Nothing is implicit, and adding a column to **return** cannot change the grouping.

This is SQL's explicit **GROUP BY** (the good part) rescued from Cypher's implicit-grouping trap, and staged so each half is one line with one name. The Cypher failure where extending **RETURN s.name** to **RETURN s.name, s.country** silently changes what is being counted is impossible: the grouping lives on its own line, untouched by the return list. The identity-versus-name trap from Part I is also closed by spelling: **group by** **sup**s can never mean "by name", and **group by** **sup**s.name says exactly what it does.

3.5 Canonical forms

New feature: one way to say each thing

For every operation there is exactly one spelling. No **WHERE** clause vs pattern-property shorthand, no three ways to count, no optional syntax. If two agents answer the same question correctly, they produce byte-identical queries.

Cypher offers `WHERE p.launch_year > 2024` and `(p:Product {launch_year: ...})` styles, plus several aggregation spellings; every choice point is a place for the model to wobble and for the grader to mis-score. Canonicity also makes the eval harness sharper (string-level checking becomes meaningful) and makes cached plan lookup trivial. The cost, verbosity in corner cases, is accepted deliberately: this is the Anka trade, removing degrees of freedom to remove errors.

3.6 Verify before execute

New feature: queries are plans; plans are checked

Every submitted query is validated as a whole against the live schema and graph statistics before anything runs: classes exist, roles exist and connect compatible classes, types line up, budgets are sane. Ill-formed plans return corrective errors; nothing partial ever executes.

This adopts the plan-verify-execute result: building a whole traversal plan and verifying it against the actual graph beats per-hop agent loops by 10 to 50% at 3 to 13 times lower cost [arXiv 2507.08945]. The line-oriented named-step syntax is what makes verification precise: each line is a typed claim (“**parts** is a set of **part**”), so the checker can point at the exact line where the plan goes wrong, in the same style as the error example above.

3.7 Token budgets and continuation

New feature: budget as a first-class clause

`budget 2000 tokens` caps the serialized result. Overflow truncates at a clean boundary and returns a continuation handle. Unstated budgets get a default; results never silently flood the context window.

```
results: 20 of 47 suppliers (budget hit)
...rows...
truncated: 27 more. resume with: continue @c81f
```

```
continue @c81f budget 1500 tokens
```

No existing query language knows the consumer has a context window. `LIMIT` bounds rows, not tokens; a row with a huge property blows the context anyway. The budget clause plus the declared-count header (“20 of 47”) gives the model an honest, checkable view of what it is seeing, per the redundancy-as-self-check principle.

3.8 The result format is part of the language

New feature: incident encoding, relevance-ordered, counts declared

Results print each node once, with its relevant edges gathered immediately around it, most relevant material first, counts declared up front, truncation always explicit.

A subgraph answer to “show me the lithium cell’s context”:

```

nodes: 3 detailed + 2 referenced, edges: 4, complete
part "lithium cell" {unit_cost: 4.20}
  supplied_by -> supplier "VoltaChem"
  supplied_by -> supplier "Ionix"
  uses <- product "PowerBank Pro" {launch_year: 2025}
  uses <- product "SolarCharger X" {launch_year: 2023}
supplier "VoltaChem" {country: "DE"}
supplier "Ionix" {country: "KR"}

```

The header distinguishes *detailed* nodes (printed with properties and gathered edges) from *referenced* nodes (named only as edge endpoints, here the two products); both counts are declared so the model can verify what it received. The choices are evidence-backed, stated carefully: encoding choice alone swings graph-task accuracy by tens of points, with incident-style encodings winning on structural tasks such as degree and neighbourhood questions [arXiv 2310.04560], while KG-LLM-Bench finds structured formats competitive on accuracy and token costs varying up to 5x between formats [arXiv 2504.07087]; keeping related edges textually adjacent counters the up-to-6x cross-referencing collapse of Lost-in-Distance [arXiv 2410.01985]; and the declared header is the self-check invariant. Incident style is the working choice for its adjacency locality and token economy, and the format is explicitly re-tested in the eval harness rather than assumed. Results stay small enough to reason over by construction: budgets cap output and truncation is always explicit, never silent.

4 Part III: what no old language has at all

Everything in Part II improves on features the old languages have. This part is the write-side surface, which no existing query language offers agents in any safe form.

4.1 Structural writes with receipts

```

assert part {name: "graphene sheet", unit_cost: 12.0}
  source doc:ingest-2026-08-24/p3 as gs
assert edge supplied_by(item: gs, source: voltachem)
  source doc:ingest-2026-08-24/p3

```

Every write states its provenance (**source** ...) and returns a **receipt**:

```

receipt: created part gs = #p-88231 at @t-99841
  guards: schema ok, class invariants ok
  dup candidates: 1
    #p-71002 part "graphene sheets" score 0.94
    shared: supplied_by -> VoltaChem; unit_cost 12.0 vs 11.8
  resolve with: merge / distinct

```

The receipt is the system’s half of the conversation. Old languages return “1 row affected”; here every write comes back with what changed, which guards ran, and, critically, any duplicate candidates the write triggered. The agent’s next action is right there in the receipt.

4.2 Deduplication: system detects, agent resolves

```
merge gs, #p-71002 prefer newest
```

```
distinct gs, #p-71002 reason "sheet vs pre-cut sheets, different SKUs"
```

The engine runs detection continuously (cheap blocking keys at write time, embedding similarity asynchronously) and exposes candidates as ordinary queryable data:

```
find dup_candidates where class = supplier order by score as dups
return dups limit 10
```

`merge` carries an explicit reconciliation policy (`prefer newest` is the default; `prefer source` <provenance> and explicit values exist). Merge semantics: one node is designated the *survivor* (by default the older, more-referenced one), the absorbed node's id becomes a permanent alias of the survivor so existing references never dangle, and lineage records the full pre-merge state of both. A merge is therefore unwindable from lineage alone. `distinct` is a durable “these are different” assertion that suppresses the pair from future candidate lists. Old-language equivalent: none. In SQL or Cypher, dedup is an external ETL job the agent never sees; in the memory systems it is automatic and silently wrong sometimes. Here it is a visible, resolvable work queue.

4.3 Granularity: blob, refine, compact

An agent ingests a supplier price table. Storing it row-by-row immediately would be slow and lossy; storing it as one node forever makes it unqueryable. Both grains are legal, and moving between them is a verb:

```
assert table_blob {title: "EU supplier prices 2026",
  payload: attach:csv-4471} source doc:report-2026-08 as prices
```

Later, when queries start needing the rows:

```
refine prices into part
  with {name: col "component", unit_cost: col "eur_unit"}
  as price_parts
```

```
receipt: refined #b-2210 -> 142 part nodes at @t-99902
  lineage: each new node carries origin #b-2210
  dup candidates: 17 (queued per class)
  blob state: retained as lineage parent (cold)
```

And the inverse, when detail has stopped paying for itself:

```
compact old_quotes as summary_quote
  {period: "2024", mean_cost: avg(old_quotes.unit_cost)}
```

Nodes carry a granularity state (`blob`, `composite`, `atom`); classes declare which states they allow; lineage makes every refinement traceable back to its source blob and every compact reversible in principle. Old-language equivalent: none. Granularity in SQL is a schema migration; in graph databases it is a hand-written ETL rewrite. Here it is two verbs the agent holds.

4.4 Health and failure nodes

The graph's own damage report is queryable data in the same language:

```
find nodes where health.loss > 0.8 order by query_traffic as worklist
return worklist.class, worklist.health, worklist.lineage limit 10
budget 1500 tokens
```

Four named subscores feed **health**: **health.loss** (construction-loss signals: extraction disagreement, property instability, contradiction density; the LossMetricKG metrics computed incrementally), **health.query** (blame from failed or flagged queries), **health.structure** (supernodes, orphans, grain mismatches), **health.staleness** (temporal decay, contradicted-by-newer). Agents feed the query signal explicitly:

```
flag #p-88231 reason "returned as Li-ion cell supplier, actually LiFeP04"
```

Maintenance is not a mode: the repair verbs are the same **merge**, **refine**, **retire**, **assert** used everywhere else, now pointed at the worklist. **retire** deserves a note: it is temporal invalidation, not deletion; the node stops being current but remains queryable historically with **after**-style time scoping.

```
retire #s-1120 reason "acquired by VoltaChem 2026-05, see #s-0071"
```

4.5 Schema evolution: derive class

```
derive class broker from supplier
with {takes_inventory: bool}
```

```
receipt: class broker provisional at @t-99950
  quota: 500 instances; promotion needs 50 stable + dedup review
  note: similar existing class "distributor" (score 0.81), review advised
```

Agents extend the schema through the language, gated: provisional status, instance quotas, dedup against existing classes, promotion after stability. On promotion the schema-closed grammar regenerates, and every agent's decoder learns the new word. The schema is data too: **schema** introspects it, and **find class where ...** queries it like anything else.

5 A construction session, end to end

An agent processes one supplier report. Abridged transcript; > lines are the agent, boxes are the system.

```
> schema
```

```
classes: product{name, launch_year} part{name, unit_cost}
         supplier{name, country}
edges: uses(whole: product, component: part)
       supplied_by(item: part, source: supplier)
```

```
> assert supplier {name: "Ionix Co", country: "KR"}  
  source doc:report-08/p1 as ionix
```

```
receipt: created supplier ionix = #s-3401  
dup candidates: 1  
  #s-0954 supplier "Ionix" score 0.97; shared country KR
```

```
> merge ionix, #s-0954 prefer newest  
> assert table_blob {title: "Ionix price list Q3"}  
  source doc:report-08/p4 as pricelist
```

```
receipt: merged -> #s-0954 (lineage keeps both)  
receipt: created table_blob pricelist = #b-5590
```

Weeks later, a maintenance agent:

```
> find nodes where health.structure > 0.7 as worklist
```

```
results: 1 of 1, complete  
table_blob #b-5590 "Ionix price list Q3"  
  signal: grain mismatch, 214 traversals/day into blob payload
```

```
> refine #b-5590 into part  
  with {name: col "item", unit_cost: col "unit_eur"} as rows
```

```
receipt: refined -> 38 part nodes, lineage kept  
dup candidates: 12 queued
```

The loop closes: writes surface duplicates, health surfaces grain problems, the same small verb set fixes both, and every step is traceable through lineage to the document it came from.

6 Feature map: old world to new

Old-language feature or failure	What the new language does
SQL explicit <code>GROUP BY</code>	Kept, and staged into its own named line
SQL join-condition errors	Gone: hops name schema roles, not key pairs
Cypher path readability	Kept as <code>follow</code> lines, one hop per line
Cypher arrows / reversed direction	No direction glyphs; wrong role type-checked where endpoint classes differ
Cypher implicit grouping	Inexpressible: grouping is its own line
Label/property hallucination	Unemittable under schema-closed decoding
SPARQL uniform patterns	Echoed: every line is verb-object-name
SPARQL URI ceremony	Dropped; names come from the schema
Gremlin composable steps	Kept, with mandatory named intermediates
Gremlin chaining confusion	Gone: no chains exist
GQL per-query path semantics	Simplified: one fixed default semantics
<code>LIMIT</code> rows	Plus <code>budget</code> tokens and continuation
“1 row affected”	Receipts: guards, dup candidates, lineage
External ETL dedup	In-language: candidates queryable, <code>merge/distinct</code>
Schema migrations	In-language, gated: <code>derive class</code> , <code>refine</code> , <code>compact</code>
No health concept	<code>health.*</code> subscores queryable; <code>flag</code> feedback

Open decisions affecting this syntax (verb tiering, exact role-naming rules, semantics defaults, budget defaults) are decisions 1–14 in the companion design briefing; everything shown here follows that document’s recommended options. The name is TBD; nothing in the syntax depends on it.